

HƯỚNG DẪN THỰC HÀNH BUỔI 8

Seeder, Factory & Query Nâng cao

Dự án: phu-xuan-blog · Laravel 10 · Môn IT3042 – Đại học Phú Xuân

MỤC TIÊU THỰC HÀNH BUỔI 8



Sau buổi thực hành, sinh viên sẽ:

- Viết Factory đầy đủ cho model Post, User, Comment với Faker
- Điều phối Seeder theo đúng thứ tự phụ thuộc (Users → Posts → Comments)
- Định nghĩa Local Scope (published, popular, recent) và dùng trong Controller
- Viết Accessor (readingTime) và Mutator (auto-slug từ title)
- Thực hành chunk(), when(), withCount() với dữ liệu 100+ records
- ★ Hoàn thiện bài kiểm tra giữa kỳ – nộp GitHub link

YÊU CẦU TRƯỚC KHI BẮT ĐẦU



Kiểm tra trước khi làm lab

- ✓ Project phu-xuan-blog chạy được: php artisan serve → localhost:8000
- ✓ Database đã có bảng: users, posts, categories, tags, post_tag, comments
- ✓ Các Model: User, Post, Category, Tag, Comment đã tồn tại
- ✓ Quan hệ Buổi 7 đã hoạt động: Post->comments, Post->tags, Post->category
- ✓ Git đã init, có remote origin trở vào GitHub repository

LAB 1: Viết Factory & Seeder cho phu-xuan-blog



Thời gian: 25 phút



Mục tiêu: Tạo bộ dữ liệu mẫu thực tế với Factory + Faker + Seeder điều phối

Bước 1: Tạo các Factory bằng Artisan

```
# Chạy từng lệnh, quan sát file được tạo ra
php artisan make:factory UserFactory --model=User
php artisan make:factory PostFactory --model=Post
php artisan make:factory CategoryFactory --model=Category
php artisan make:factory CommentFactory --model=Comment
```

```
# Kiểm tra file đã được tạo
```

```
ls database/factories/
```

```
# Kết quả mong đợi:
```

```
# CategoryFactory.php  CommentFactory.php  PostFactory.php  UserFactory.php
```

Bước 2: Viết CategoryFactory

Mở file database/factories/CategoryFactory.php và viết:

```
<?php
```

```
namespace Database\Factories;
```

```
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Str;

class CategoryFactory extends Factory
{
    public function definition(): array
    {
        $name = $this->faker->unique()->randomElement([
            'Công nghệ', 'Lập trình', 'Thiết kế', 'Kinh doanh',
            'Du lịch', 'Ẩm thực', 'Sức khỏe', 'Giáo dục',
            'Thể thao', 'Giải trí', 'Khoa học', 'Văn hóa',
        ]);

        return [
            'name' => $name,
            'slug' => Str::slug($name),
            'description' => $this->faker->sentences(2, true),
        ];
    }
}
```

Bước 3: Viết PostFactory đầy đủ

Mở file database/factories/PostFactory.php:

```
<?php

namespace Database\Factories;

use App\Models\User;
use App\Models\Category;
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Str;

class PostFactory extends Factory
{
    public function definition(): array
    {
        $title = $this->faker->sentence(rand(4, 8));

        return [
            'title' => rtrim($title, '.'),
            'slug' => Str::slug($title) . '-' . $this->faker->
>unique()->numberBetween(1, 99999),
            'content' => implode('\n\n', $this->faker->
>paragraphs(rand(3, 6))),
            'excerpt' => $this->faker->sentences(2, true),
            'user_id' => User::inRandomOrder()->first()->id ?? 1,
            'category_id' => Category::inRandomOrder()->first()->id ?? 1,
            'status' => $this->faker->randomElement(['draft',
'published', 'published', 'published']),
            'published_at' => $this->faker->optional(0.75)-
>dateTimeBetween('-1 year', 'now'),
            'view_count' => $this->faker->numberBetween(0, 50000),
        ];
    }

    // State: bài viết published
}
```

```

    public function published(): static
    {
        return $this->state(fn (array $attr) => [
            'status' => 'published',
            'published_at' => $this->faker->dateTimeBetween('-6 months',
'now'),
        ]);
    }

    // State: bài viết draft
    public function draft(): static
    {
        return $this->state(fn (array $attr) => [
            'status' => 'draft',
            'published_at' => null,
        ]);
    }
}

```

Bước 4: Viết CommentFactory

```

<?php

namespace Database\Factories;

use App\Models\User;
use App\Models\Post;
use Illuminate\Database\Eloquent\Factories\Factory;

class CommentFactory extends Factory
{
    public function definition(): array
    {
        return [
            'body' => $this->faker->paragraph(),
            'user_id' => User::inRandomOrder()->first()->id ?? 1,
            'post_id' => Post::inRandomOrder()->first()->id ?? 1,
            'is_approved' => $this->faker->boolean(80), // 80% approved
        ];
    }
}

```

Bước 5: Viết các Seeder riêng

Tạo các seeder:

```

php artisan make:seeder UserSeeder
php artisan make:seeder CategorySeeder
php artisan make:seeder PostSeeder
php artisan make:seeder CommentSeeder

```

Nội dung database/seeder/UserSeeder.php:

```

public function run(): void
{
    // Tạo admin account cố định
    \App\Models\User::create([
        'name' => 'Admin Blog',
        'email' => 'admin@phu-xuan-blog.test',
        'password' => bcrypt('password'),
    ]);
}

```

```
});

// Tạo 9 user ngẫu nhiên
\App\Models\User::factory()->count(9)->create();
}
```

Nội dung database/seeder/PostSeeder.php:

```
public function run(): void
{
    // 70 bài published
    \App\Models\Post::factory()->published()->count(70)->create();

    // 20 bài draft
    \App\Models\Post::factory()->draft()->count(20)->create();

    // 10 bài published nổi bật (view_count cao)
    \App\Models\Post::factory()->published()->count(10)->create([
        'view_count' => rand(10000, 50000),
    ]);

    // Gán tags ngẫu nhiên cho tất cả posts
    \App\Models\Post::all()->each(function ($post) {
        $tagIds = \App\Models\Tag::inRandomOrder()->limit(rand(1, 4))-
        >pluck("id");
        $post->tags()->sync($tagIds);
    });
}
```

Bước 6: Điều phối DatabaseSeeder

Mở database/seeder/DatabaseSeeder.php – thứ tự call RẤT QUAN TRỌNG:

```
public function run(): void
{
    // 1. Users trước (Posts cần user_id)
    $this->call(UserSeeder::class);

    // 2. Categories (Posts cần category_id)
    $this->call(CategorySeeder::class);

    // 3. Tags (Posts cần tags để sync pivot)
    $this->call(TagSeeder::class);

    // 4. Posts (cần users + categories + tags đã có)
    $this->call(PostSeeder::class);

    // 5. Comments (cần users + posts đã có)
    $this->call(CommentSeeder::class);
}
```

Bước 7: Chạy migration fresh + seed

```
# Reset hoàn toàn DB rồi migrate lại + seed
php artisan migrate:fresh --seed

# Nếu thành công, output sẽ giống:
INFO Preparing database.
Dropping all tables ..... 48ms DONE
```

```
INFO Running migrations.
2024_01_01_000000_create_users_table .... 15ms DONE
...

INFO Seeding database.
UserSeeder ..... 42ms DONE
CategorySeeder ... 15ms DONE
TagSeeder ..... 8ms DONE
PostSeeder ..... 3234ms DONE
CommentSeeder .... 1200ms DONE
```

✓ CHECKPOINT: Lab 1 hoàn thành

- ✓ php artisan migrate:fresh --seed chạy không lỗi
- ✓ Mở TablePlus/phpMyAdmin: bảng posts có ~100 records
- ✓ Bảng users có 10 records; categories và tags có dữ liệu
- ✓ Bảng post_tag (pivot) có nhiều records – tags đã được gán
- ✓ Bảng comments có dữ liệu ngẫu nhiên

Vấn đề gặp phải	Nguyên nhân có thể	Cách khắc phục
Call to a member function id() on null (trong Factory)	User::inRandomOrder()->first() trả về null vì bảng users trống	Đảm bảo UserSeeder chạy TRƯỚC PostSeeder trong DatabaseSeeder
SQLSTATE[23000] Duplicate entry for slug	Faker tạo slug trùng khi title ngẫu nhiên giống nhau	Thêm ->numberBetween() vào slug: Str::slug(\$title) . '-' . rand(1,99999)
Cannot call factory() method – trait không tồn tại	Model chưa dùng trait HasFactory	Thêm use Illuminate\Database\Eloquent\Factories\HasFactory; vào Model

LAB 2: Scope, Accessor & Query Nâng cao

🕒 Thời gian: 25 phút

🎯 Mục tiêu: Viết Local Scope, Accessor, dùng chunk() và when() trong Controller

Bước 1: Thêm Local Scope vào Post Model

Mở app/Models/Post.php và thêm các scope sau:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Post extends Model
{
    use SoftDeletes;
```

```

protected $fillable = [...];

protected $casts = [
    'published_at' => 'datetime',
    'view_count'   => 'integer',
];

// — Local Scopes —————

// Scope: chỉ lấy bài đã published
public function scopePublished(Builder $query): Builder
{
    return $query->where('status', 'published')
        ->whereNotNull('published_at')
        ->where('published_at', '<=', now());
}

// Scope: sắp xếp theo phổ biến nhất
public function scopePopular(Builder $query): Builder
{
    return $query->orderByDesc('view_count');
}

// Scope: bài viết gần đây (mặc định 7 ngày)
public function scopeRecent(Builder $query, int $days = 7): Builder
{
    return $query->where('published_at', '>=', now()->subDays($days));
}

// Scope: lọc theo danh mục
public function scopeOfCategory(Builder $query, int $categoryId): Builder
{
    return $query->where('category_id', $categoryId);
}

// — Accessor —————

// Thời gian đọc ước tính (200 từ/phút)
protected function readingTime(): Attribute
{
    return Attribute::make(
        get: fn () => max(1, (int) ceil(str_word_count(strip_tags($this->content)) / 200)) . " phút đọc",
    );
}

// — Relationships —————
public function user()      { return $this->belongsTo(User::class); }
public function category() { return $this->belongsTo(Category::class); }
public function tags()     { return $this->belongsToMany(Tag::class); }
public function comments() { return $this->hasMany(Comment::class); }
}

```

Bước 2: Cập nhật PostController dùng Scope + when()

Mở app/Http/Controllers/PostController.php, cập nhật method index():

```
use App\Models\Post;
```

```

use Illuminate\Http\Request;

public function index(Request $request)
{
    $posts = Post::query()
        ->published() // Local Scope
        ->with(["user", "category", "tags"]) // Eager loading
        ->withCount("comments") // Đếm comments
        ->when($request->search, function ($q, $search) {
            $q->where('title', 'like', "%{$search}%");
        })
        ->when($request->category_id, function ($q, $catId) {
            $q->ofCategory($catId); // Dùng scope với tham số
        })
        ->when($request->sort === 'popular', function ($q) {
            $q->popular(); // Scope popular
        }, function ($q) {
            $q->orderByDesc('published_at'); // Default: mới nhất
        })
        ->paginate(10)->withQueryString();

    $categories = \App\Models\Category::all();

    return view("posts.index", compact("posts", "categories"));
}

```

Bước 3: Cập nhật View sử dụng dữ liệu mới

Mở resources/views/posts/index.blade.php – thêm hiển thị số comment và reading_time:

```

@foreach ($posts as $post)
    <div class="post-card">
        <h2><a href="{{ route("posts.show", $post) }}">{{ $post->title }}</a></h2>

        <div class="meta">
            <span>{{ $post->user->name }}</span>
            <span>{{ $post->published_at?->format('d/m/Y') }}</span>
            <span>{{ $post->reading_time }}</span>    {{-- Accessor --}}
            <span>{{ $post->comments_count }} bình luận</span>    {{--
withCount --}}
        </div>

        <p>{{ $post->excerpt }}</p>

        <div class="tags">
            @foreach ($post->tags as $tag)
                <span class="tag">{{ $tag->name }}</span>
            @endforeach
        </div>
    </div>
@endforeach

{{-- Pagination links --}}
{{ $posts->links() }}

```

Bước 4: Thêm Form tìm kiếm vào View

Thêm form filter phía trên danh sách:

```

<form method="GET" action="{{ route('posts.index') }}">
    <input type="text" name="search" value="{{ request('search') }}"
placeholder="Tìm kiếm...">

    <select name="category_id">
        <option value="">Tất cả danh mục</option>
        @foreach ($categories as $cat)
            <option value="{{ $cat->id }}" {{ request('category_id') == $cat-
>id ? "selected" : "" }}>
                {{ $cat->name }}
            </option>
        @endforeach
    </select>

    <select name="sort">
        <option value="newest" {{ request('sort') === "newest" ? "selected" :
"" }}>Mới nhất</option>
        <option value="popular" {{ request('sort') === "popular" ? "selected"
: "" }}>Phổ biến nhất</option>
    </select>

    <button type="submit">Tìm</button>
    <a href="{{ route('posts.index') }}">Xóa filter</a>
</form>

```

Bước 5: Test trong Tinker – xác minh Scope hoạt động

```

php artisan tinker

# Test scope published()
App\Models\Post::published()->count()
# Phải trả về số < 100 (chỉ bài published)

# Test scope popular()
App\Models\Post::published()->popular()->first()->view_count
# Phải trả về số view_count lớn nhất

# Test Accessor reading_time
$p = App\Models\Post::first();
$p->reading_time
# Output: "X phút đọc"

# Test kết hợp scope + eager loading + withCount
App\Models\Post::published()->with("user")->withCount("comments")-
>paginate(10)
# Xem có N+1 không? Chỉ nên có 3 queries tổng cộng

```

✓ CHECKPOINT: Lab 2 hoàn thành

- ✓ http://localhost:8000/posts hiển thị danh sách 10 bài/trang (paginate)
- ✓ Form search: nhập từ khóa → lọc được bài viết
- ✓ Dropdown danh mục hoạt động
- ✓ Dropdown sort "Phổ biến nhất" sắp xếp đúng theo view_count
- ✓ Card bài viết hiển thị: reading_time, số comments, tags
- ✓ Không có N+1 query (Laravel Debugbar hiển thị ≤ 4 queries/trang)

Vấn đề gặp phải	Nguyên nhân có thể	Cách khắc phục
reading_time hiển thị null hoặc không xuất hiện	Accessor chưa được append hoặc method name không khớp chuẩn camelCase	Kiểm tra method tên 'readingTime()' trong Model; thêm protected \$appends = ['reading_time']
scope published() không lọc được – vẫn hiện bài draft	Tên method scope viết sai hoặc gọi sai (viết hoa)	Trong Model: scopePublished(); khi gọi: Post::published() (không có tiền tố scope)
withQueryString() không tồn tại (Laravel cũ)	Method withQueryString() chỉ có từ Laravel 8.x+	Dùng ->appends(request()->query()) thay thế

LAB 3: ★ KIỂM TRA GIỮA KỲ – Hoàn thiện Blog App & Nộp GitHub

🕒 Thời gian: 30 phút

🎯 Mục tiêu: Tích hợp tất cả kiến thức Phase 2 – Nộp bài để chấm điểm giữa kỳ

⚠️ ĐÂY LÀ BÀI KIỂM TRA GIỮA KỲ

- Deadline: 1 tuần sau buổi học này.
- Nộp bài bằng cách push code lên GitHub rồi điền link vào Google Sheet (giảng viên chia sẻ).
- Làm cá nhân – không copy code của bạn khác.
- Cần đảm bảo giảng viên có thể clone về + chạy php artisan migrate:fresh --seed + thấy kết quả.

Checklist bắt buộc (tối thiểu để được điểm):

#	Yêu cầu	File/Route cần kiểm tra	Điểm
1	PostFactory đủ các field: title, slug (unique), content, excerpt, status, published_at, view_count	database/factories/PostFactory.php	10
2	DatabaseSeeder gọi đúng thứ tự: User → Category → Tag → Post (≥ 100) → Comment	database/seeder/DatabaseSeeder.php	10
3	Scope published() và popular() hoạt động đúng	app/Models/Post.php	10
4	Trang /posts: hiển thị đúng 10/trang, pagination hoạt động	GET /posts	10
5	README.md: hướng dẫn clone + migrate:fresh --seed + cách test	README.md (root)	10

Yêu cầu nâng cao (cộng điểm bonus):

Tính năng	Mô tả	Điểm cộng
Search + Filter	Form tìm kiếm theo keyword, lọc category, sort (newest/popular)	+5
Accessor readingTime	Hiển thị "X phút đọc" trong card bài viết	+3
withCount comments	Hiển thị số comments trong danh sách	+2

Bước 1: Chạy kiểm tra cuối cùng trước khi nộp

```
# 1. Reset và seed lại để đảm bảo không có lỗi
```

```
php artisan migrate:fresh --seed

# 2. Chạy app và kiểm tra tay
php artisan serve
# Mở trình duyệt: http://localhost:8000/posts

# 3. Kiểm tra paginate
# Nhấn "Next" để xem trang 2, 3...

# 4. Test scope bằng Tinker
php artisan tinker
>>> App\Models\Post::published()->count() // phải > 0
>>> App\Models\Post::popular()->first()->view_count // số lớn nhất

# 5. Commit và push lên GitHub
git add .
git commit -m "feat: hoàn thành buổi 8 - Seeder, Factory, Scope, Paginate"
git push origin main
```

Bước 2: Viết README.md

Tạo hoặc cập nhật README.md ở thư mục gốc project với nội dung tối thiểu:

```
# phu-xuan-blog

Blog app xây dựng với Laravel 10 - Bài tập môn IT3042 Đại học Phú Xuân.

## Tính năng
- Quản lý bài viết (CRUD)
- Danh mục, Tag, Comments
- Tìm kiếm, lọc danh mục, sắp xếp
- Phân trang 10 bài/trang

## Cài đặt

```bash
git clone <your-repo-url>
cd phu-xuan-blog
composer install
cp .env.example .env
php artisan key:generate
Cập nhật DB_DATABASE, DB_USERNAME, DB_PASSWORD trong .env
php artisan migrate:fresh --seed
php artisan serve
```

## Test
- Truy cập http://localhost:8000/posts
- Tài khoản test: admin@phu-xuan-blog.test / password
```

Bảng test case tự kiểm tra trước khi nộp:

| Test case | Thao tác kiểm tra | Kết quả mong đợi | Kết quả thực tế |
|-----------------|----------------------------------|---|-----------------|
| Seed thành công | php artisan migrate:fresh --seed | Không lỗi, output "DONE" cho mỗi seeder | |

| Test case | Thao tác kiểm tra | Kết quả mong đợi | Kết quả thực tế |
|-----------------------|-----------------------------------|--|-----------------|
| Dữ liệu đúng số lượng | Post::count() trong Tinker | Trả về 100 (hoặc số đã config) | |
| Scope published | Post::published()->count() | Nhỏ hơn Post::count() | |
| Trang danh sách | GET /posts | Hiện 10 bài, có nút Next/Prev | |
| Tìm kiếm | GET /posts?search=laravel | Lọc đúng bài có từ "laravel" trong tiêu đề | |
| Lọc danh mục | GET /posts?category_id=1 | Chỉ hiện bài thuộc danh mục ID 1 | |
| Sort phổ biến | GET /posts?sort=popular | Bài đầu tiên có view_count cao nhất | |
| GitHub | Clone repo, chạy hướng dẫn README | App chạy được không cần hỏi thêm | |

BẢNG TỰ ĐÁNH GIÁ BUỔI 8

| Kỹ năng | Tự đánh giá | Ghi chú |
|--|---|---------|
| Viết Factory với Faker đủ fields và states | ☐ Chưa ☐ Cơ bản ☐ Thành thạo | |
| Điều phối Seeder đúng thứ tự | ☐ Chưa ☐ Cơ bản ☐ Thành thạo | |
| Định nghĩa Local Scope trong Model | ☐ Chưa ☐ Cơ bản ☐ Thành thạo | |
| Kết hợp nhiều scope + eager loading | ☐ Chưa ☐ Cơ bản ☐ Thành thạo | |
| Dùng when() để filter query linh hoạt | ☐ Chưa ☐ Cơ bản ☐ Thành thạo | |
| Viết Accessor trả về field ảo | ☐ Chưa ☐ Cơ bản ☐ Thành thạo | |
| Paginate + withQueryString() | ☐ Chưa ☐ Cơ bản ☐ Thành thạo | |
| Push code lên GitHub + viết README | ☐ Chưa ☐ Cơ bản ☐ Thành thạo | |

✓ Tóm tắt những gì đã học trong Buổi 8

- ✓ Seeder: DatabaseSeeder điều phối các seeder con theo thứ tự phụ thuộc
- ✓ Factory + Faker: blueprint sinh dữ liệu ngẫu nhiên thực tế quy mô lớn
- ✓ Factory States: .published(), .draft() để tạo variations
- ✓ Local Scope: method scopeXxx() trong Model, gọi bằng ->xxx() trong query
- ✓ Accessor: Attribute::make(get:) để tạo field ảo hoặc transform giá trị đọc
- ✓ when(): thêm điều kiện query linh hoạt, sạch code hơn if/else
- ✓ chunk(): xử lý big data an toàn, tránh hết RAM

**Preview Buổi 9 – Authentication**

- Cài đặt Laravel Breeze: đăng ký, đăng nhập, đăng xuất
- Middleware: auth, guest – bảo vệ route tạo/sửa bài
- Auth facade: lấy thông tin user đang login
- Chuẩn bị: đọc docs.laravel.com/10.x/starter-kits